Salami Assignment, Mathematical Modeling 02526 S26

Maciej Jałocha (macja@itu.dk)
William Michelsen (wimi@itu.dk)

1 Introduction

Sausages of good quality are important for the nourishment of the Danish population and other nations. However, if the quality of the sausage is poor, people risk exposure to mycotoxins from fungi as well as severe bacterial infections, which may be lethal. Even under clean manufacturing circumstances, the production process can lead to salami of varying quality, making it essential to monitor key parameters such as the meat-to-fat ratio.

To address this need, the purpose of this project is to develop a technique for the automatic determination of the amount of meat and fat in the sausage. We utilized multispectral images of salami slices recorded over a 28-day drying period. By considering each pixel as an independent continuous stochastic variable, we applied Linear Discriminant Analysis (LDA) to classify each pixel as either meat or fat. A simpler model like LDA was chosen due to the limited number of annotated pixels available for training, which helps reduce the risk of overfitting while still providing an interpretable baseline for classification.

This report documents our approach and findings. In the following sections, we describe the multispectral data and the mathematical models employed in our experiments. We then present the results of our analysis, detailing the search for the best-performing model and training day, as well as an evaluation of the classification errors. Finally, we discuss the limitations and validity of the chosen model, and conclude on the feasibility of using this approach for automatic salami quality assessment.

2 Methods and Materials

The data we used consist of multispectral images of salami in production over 28 days of drying. A multispectral salami image is composed of 19 images at different wavelengths (spectral bands), resulting in 19 measurements for each pixel. We used one image (from a different salami each time) from five of the 28 days: 1, 6, 13, 20, and 28. The images were downsampled to a spatial resolution of $514 \times 514 \times 19$ and reduced to 8-bit grayscale.

We applied both a one-dimensional and a multidimensional variant of a linear discriminant analysis (LDA) model, using the pixel values from the different spectral bands as features. Mathematically, we defined an observed pixel as a continuous stochastic variable x. For the one-dimensional model, x is a scalar representing the intensity from a single spectral band. For the multidimensional model, the pixel is represented as a feature vector $x \in \mathbb{R}^{19}$, capturing intensities across all 19 spectral bands simultaneously.

We used the LDA models to classify each observed pixel x as either meat or fat, which effectively assumes that each pixel belongs exclusively to one of the two classes. To train our model, we used associated annotated versions of each image (example in Figure 1) in which salami meat and fat have been labeled. Only parts of each salami have been annotated. The dataset also provides a mask to isolate the salami from the background, which we used to make predictions for unlabeled pixels as well.

Since the number of annotated pixels is somewhat limited, we chose a simple model such as

LDA, which requires less data and reduces the risk of overfitting. LDA also offers some robustness to violations of its assumptions, some of which we discuss later. Using this model allows us to assess the effectiveness of simpler approaches on this problem while providing interpretable results and parameters.

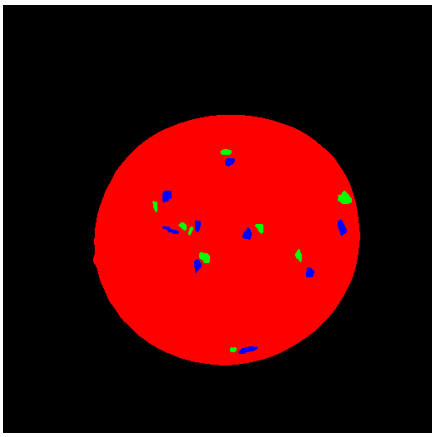


Figure 1: Example annotation. Green pixels are fat and blue pixels are meat (used for training). Red pixels belong to the salami but are unlabeled. Black pixels are background.

3 Results

We selected the best model by comparing three models, and the best training day by training on five different days. We found that the all-bands multi-dimensional LDA and day 28 performed best. We then analyzed its errors. In our analysis we used the error rate defined as the fraction of misclassified annotated pixels, and manual inspection of model annotations against ground-truth color images.

3.1 Finding the best model

First, we compare a majority baseline, a best-band one-dimensional LDA, and an all-bands multi-dimensional LDA. All parameters for the models—means, covariance (matrix), and priors—are estimated from the day 1 annotations.

Finding the best band for the one-dimensional LDA. We chose the best band for the one-dimensional LDA as the one with the lowest error rate when trained and evaluated on day 1. Band 14 achieved the lowest error rate.

Metric	Value
Mean	0.03
Std	0.1
Min ER	0.009
Max ER	0.43

Table 1: Basic statistics for a one-dimensional LDA trained and evaluated on day 1 annotations.

Comparing all-bands multi-dimensional LDA with best-band one-dimensional LDA. Multi-dimensional LDA performs better than one-dimensional LDA across all validation days (Table 2).

Validation Day	1D LDA (band 14)	m. LDA
6th	0.11	0.02
13th	0.12	0.09
20th	0.24	0.14
28th	0.18	0.06

Table 2: Error rates for the best-band (14th) one-dimensional and all-bands multi-dimensional LDA model trained on day 1 annotations and evaluated on the remaining day annotations.

The manual, visual inspection suggests the same. The one-dimensional model is prone to labeling meat pixels as fat (Figure 2).

Comparing all-bands multi-dimensional LDA with majority baseline. The all-bands multi-dimensional LDA performs better than the

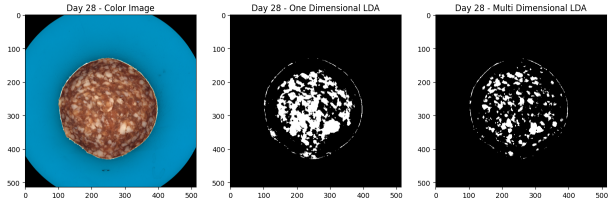


Figure 2: A comparison of the best one-dimensional and multi-dimensional LDA models. The leftmost image represents the color image of a slice of a salami on day 28th, middle one represents the one-dimensional LDA labelling and the rightmost one represents the multi-dimensional LDA labelling. White pixels represent fat and black background or meat.

majority baseline. For the training day, the majority baseline attains an error rate of 0.43. We did not evaluate the remaining days because we did not expect the baseline to perform better on unseen days than on the training day.

We therefore chose the multi-dimensional LDA for the remaining experiments, and proceed to search for the best training day.

3.2 Finding the best training day

To find the best training day, we trained on each day and evaluated on the rest. For each of the five available days, we trained a model on that day's annotations and evaluated it on the four remaining days' annotations (Figure 3).

We found that both day 13 and day 28 yielded the lowest error rates across the remaining days. Rather than choosing immediately, we performed a manual visual analysis for both models on all remaining days, which allows us to evaluate behavior in unannotated regions as well.

We observed that the model trained on day 28 is better at capturing fat in the middle of a salami without labeling additional meat as fat (Figure 4).

Therefore, we chose the multi-dimensional LDA trained on day 28 annotation as the best performing model and day setup pair.

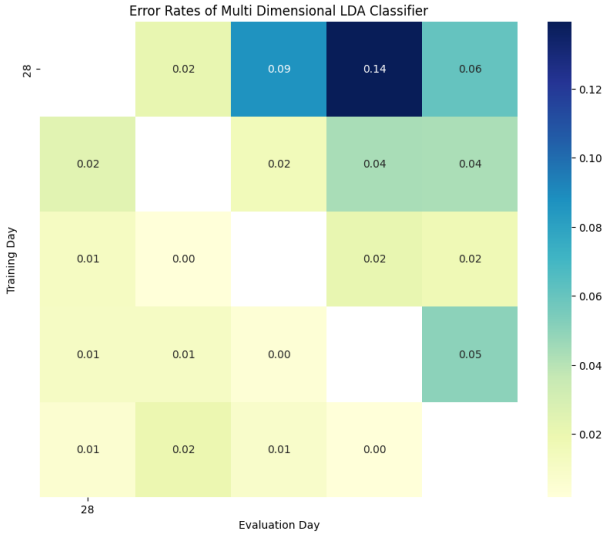


Figure 3: Error rates for multi-dimensional LDA, trained on each and evaluated on different days.

3.3 Impact of Class Priors

In the last experiment, we evaluated the impact of the class priors. We re-evaluated the multi-dimensional LDA trained on day 28 on the remaining days using a fixed prior (fat ratio = 0.3) instead of the empirical priors estimated from the annotations. To our surprise, we found no noticeable difference in error rate (differences appear only in the third decimal place). The predicted fat/meat ratio decreased slightly (Table 3), but we were unable to see a visual difference (Figure 5).

Days	Priors from data	Fat ratio = 0.3
1st	28.76%	28.27%
6th	31.33%	30.67%
13th	29.82%	29.07%
20th	22.87%	22.44%

Table 3: Ratio of fat-labeled pixels by LDA with priors estimated from the data and by LDA with prior fat ratio = 0.3.

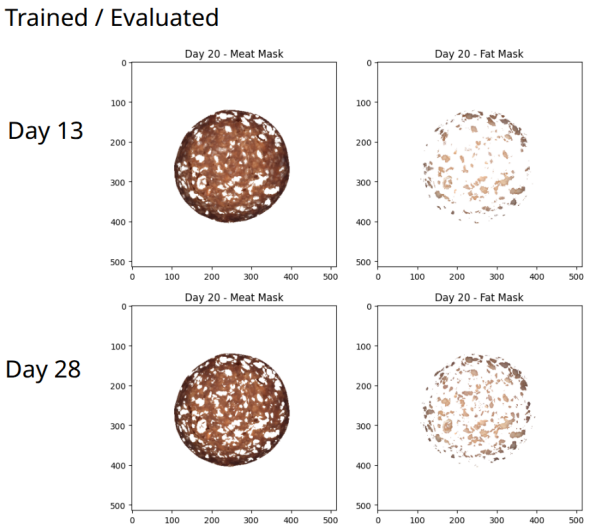


Figure 4: Visual comparison of multi-dimensional LDA trained on days 13 and 28 and evaluated on day 20th

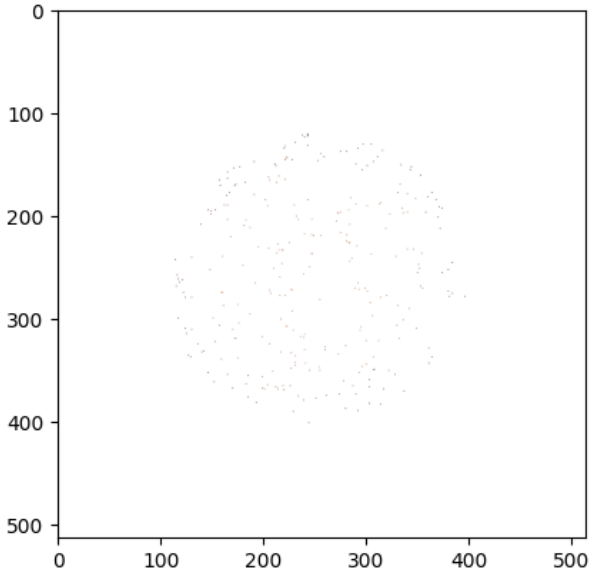


Figure 5: Pixels that were labeled as fat by an LDA with priors estimated from the data but not by an LDA with prior fat ratio = 0.3.

3.4 Analysis of the Errors

We evaluated the sources of errors for the all-bands multi-dimensional LDA (Figure 6) trained on day 28 (best model). Despite its strong performance on the annotated data, the scarcity of validation data motivated an additional manual visual analysis.

We analyzed the model's predictions on the four validation-day color images. We observed that the model sometimes classifies meat as fat near the edges on day 1 (Figure 7) and day 6.

We also observed a few cases on day 13 (Figure 8) and day 20 where meat *could be* mislabeled as fat.

4 Discussion

The LDA model has notable shortcomings. Primarily, it assumes equal covariance matrices between the meat and fat classes. Our visualizations of the data show this assumption is likely violated (Figure 9). Given the lack of non-linear terms in the model's discriminant function, it also means the problem ends up being modeled with linear decision boundaries. Given the potentially lethal risks associated with poor-quality salami, we expect that even with our current low error rates, they remain too high for commercial deployment. The seriousness of the problem may therefore necessitate a model trained on more data, capable of allowing non-linear decision boundaries to adequately model the data. This may also help lessen some misclassifications occurring in ambiguous areas like those marked in Figure 8. Therefore, while the solution(s) here are a proof of concept, we do not expect them to be reliable enough for commercial deployment.

As mentioned, training on day 28 provides the best performance, while day 13 provides the second best. Arguments can be made for both in terms of which model to actually use in a practical setting, if one were to train on one specific day. When trained on day 28 the model achieves the lowest error rate. Based on a qualitative analysis of the predictions, we believe this model also displays the most reasonable behaviour on non-annotated pixels. Lastly, if the ultimate goal of the system is to verify the quality of the final

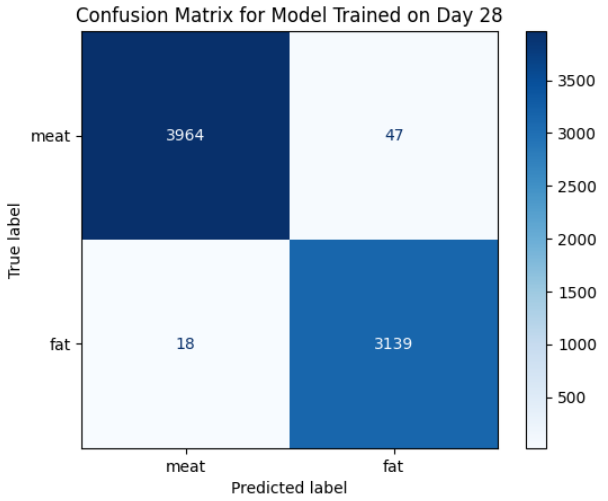


Figure 6: Confusion matrix for all-bands multi-dimensional LDA trained on day 28 annotations and evaluated on 4 remaining day annotations.

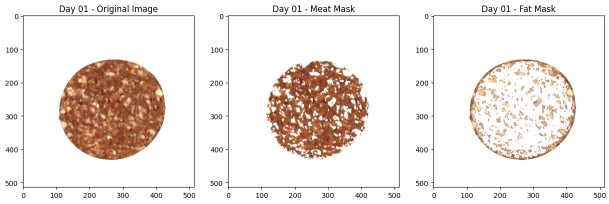


Figure 7: Annotations made by multi-dimensional LDA trained on day 28 annotations, applied to the day 1 color image.

product before it reaches consumers, the data from day 28 would be the most relevant. The model, when trained on day 13, achieves only slightly worse performance. We expect training on this day to also be a reasonable choice in practice as the data is likely more representative of the entire dataset due to its temporal centrality.

Another relevant concern is whether the data is properly representative of the problem. As previously stated, the images are not entirely annotated. This may affect estimated ratios of meat and fat, which is highly relevant for estimating the quality of the salami. Furthermore, we suspect it affects priors of the LDA models and their results. For instance, after a visual inspection of predictions made on non-annotated pixels, we found that the results appear questionable, even

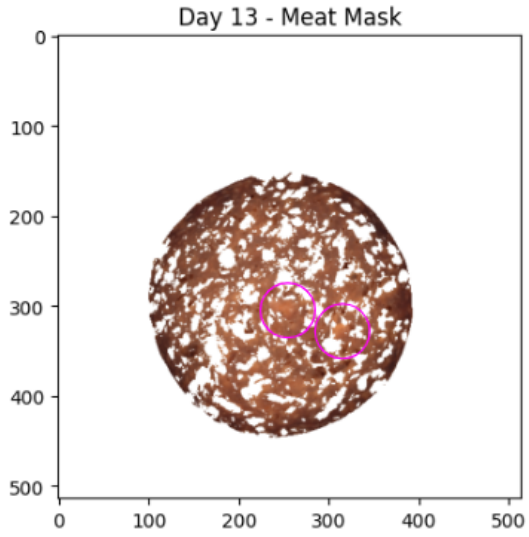


Figure 8: Day 13 prediction by multi-dimensional LDA trained on day 28 annotations. Two unclear (meat or fat) areas are marked.

for the model trained on day 28 (Figure 8). This is especially apparant when compared to colored versions of the multispectral images. We also suspect that we may be underestimating the true error rates. This is due to our suspicion that only relatively unambiguous pixels have been annotated, which may mean that we are training and predicting on the easiest cases. This also puts the modeling assumption that a pixel is exclusively either meat or fat into question. We expect one may get more nuanced, accurate results by modeling the response variable, or the problem as a whole, differently.

5 Conclusions

We found that automatic fat/meat ratio detection from spectral images is feasible. Moreover, it can be achieved with an interpretable, lightweight model using little data. We believe that our solution could already be used to assist human annotators in determining fat/meat ratios in salamis, and that additional research could yield near-perfect automatic solutions.

A Appendix

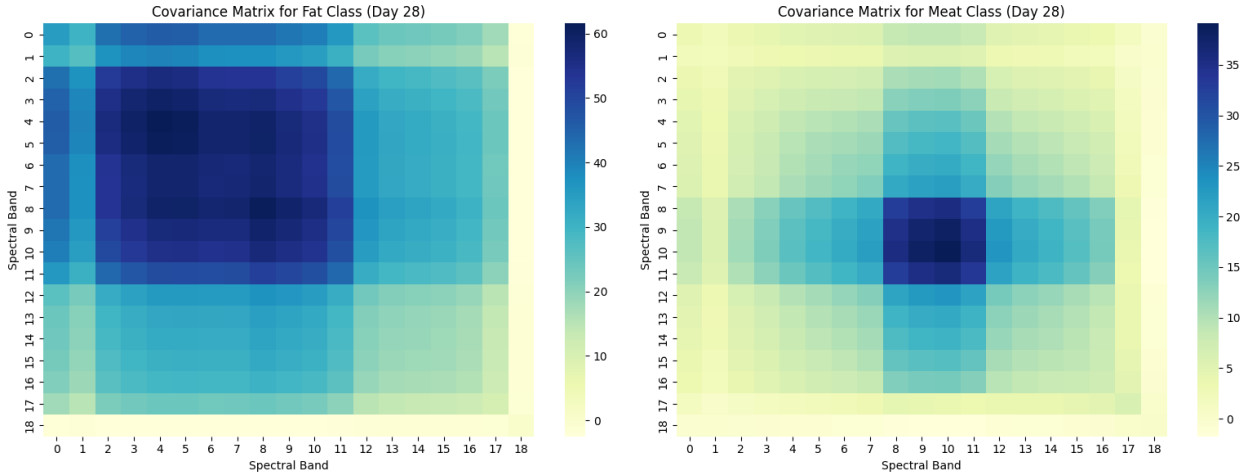


Figure 9: Covariance matrices, estimated on day 28 annotations for fat and meat classes.

A.1 Use of Generative AI

The analysis code has been almost completely written by us, with Github Copilot used for debugging. The report was written by us, we used OpenAI's Prism for proofreading and formatting.

A.2 Analysis code

We used the *helpFunctions* from coursepage. We fixed *setImagePix* ihy removing the last line before returng performing *uint*8 casting.

```
# %% [markdown]
#
#
# ### Describtion of the problem:
# #### Look at the images and consider how to distinguish between fat and meat.
# In colour - meet is red, fat is white.
# #### Which visual changes are apparant in the salami over the 28 days?
# It gets darker around the edges and the white stripe around the corner shows up. Also, it seems
    that fat is more apparent, the "bulbs" are also larger (they appear in larger chunks). The
    surface seems more smooth. The edge is less circullar. I also think it could have gotten
    smaller?
# #### How can these changes be measured from the images?
# * We could look into color distributions, color ratios. of red to white
# * look for single big white chunks by applying circular filters?
# * the same for black around the edges
# * Apply some filters to find these non-smooth areas (very apparent in the beggining).
# * measure the area and ovalness of the salami
# #### Load a multi spectral image in MATLAB or PYTHON and investigate the data.
# BELOW

# %%
```

```
import helper_functions.helpFunctions as hf

dirIn = "./data/"

multiIm, annotationIm = hf.loadMulti(
    "multispectral_day20.mat", "annotation_day20.png", dirIn
)

# %%
import matplotlib.pyplot as plt

plt.imshow(multiIm[:, :, 13], cmap="gray")

# %%
import numpy as np

fatPix, fatR, fatC = hf.getPix(multiIm, annotationIm[:, :, 1])
meatPix, meatR, meatC = hf.getPix(multiIm, annotationIm[:, :, 2])

x = np.arange(1, 20)
width = 0.4
plt.figure()
plt.bar(x, np.var(meatPix, 0), width, color="b", label="meat")
offset = width
plt.bar(x + offset, np.var(fatPix, 0), width, color="r", label="fat")
plt.xticks(x + width / 2, x)
plt.legend()
plt.show()

# %% [markdown]
# So, that's after reading the section
# ### Formulation of the mathematical model:
# #### Describe the mathematical model
# The goal of the model is to classificy the pixel and meat and fat.
#
# The model simply assumes that the samples are normally distributed with different means and
    standard deviations.
# #### Why do we expect it to work?
# * Assuming normal distrbituion: usually biological data is normally distributed.
# * We assume the means should be different - from looking at the data
# #### What are the limitations of the model?
# * The assumption of the same standard deviation is simply wrong, given the plot.
# * The assumption of the independence - It is false, because of the location dependence. This is
    not a noise, if a pixel is white, then likely its surrounding is white as well. I do not think
    it breaks down the model, but is a missed opportunity. However, it should hold across the
    bigger window.

# %% [markdown]
# ### Analysis of the data
#
#

# %% [markdown]
# #### Investigate the data
# * The histgram function is not normalized, so simply more pixels = higher; when comparing both of
     them.

# %%
multiIm, annotationIm = hf.loadMulti(
```

```
    "multispectral_day20.mat", "annotation_day20.png", dirIn
)
# %%
# Show image of spectral band 7
plt.imshow(multiIm[:, :, 6])
plt.show()

# %%
# annotationIm is a binary image with 3 layers
annotationIm.shape

# %%
# In each layer we have a binary image:
# 0 - background with salami
# 1 fat annotation
# 2 meat annotation

# Here we show the meat annotation
plt.imshow(annotationIm[:, :, 0])
plt.show()

# %%
# The function getPix extracts the multi spectral pixels from the annotation
# Here is an example with meat- and fat annotation
fatPix, fatR, fatC = hf.getPix(multiIm, annotationIm[:, :, 1])
meatPix, meatR, meatC = hf.getPix(multiIm, annotationIm[:, :, 2])

# %%
print(fatPix)
# %%
# Here we plot the mean values for pixels with meat and fat respectively
plt.plot(np.mean(meatPix, 0), "b")
plt.plot(np.mean(fatPix, 0), "r")
plt.show()


# %%
# %%
# The function setImagePix produces a colour image
# where the pixel coordinates are given as input
# Load RGB image
imRGB = plt.imread(dirIn + "color_day20.png")

# Pixel coordinates for the fat annotation
fatPix, fatR, fatC = hf.getPix(multiIm, annotationIm[:, :, 1])

# Concatenate the pixel coordinates to a matrix
pixId = np.stack((fatR, fatC), axis=1)

# Make the new images
rgbOut = hf.setImagePix(imRGB, pixId)
plt.imshow(rgbOut, cmap="gray")
plt.show()


# %%
rgbOut


```

```
# %% [markdown]
# #### Spectral distribution

# %%
n_layers = 19
fig, ax = plt.subplots(n_layers, 1, figsize=(10, 2 * n_layers))
hist_pairs = []
for layer_id in range(0, n_layers):
    h = hf.showHistograms(
        multiIm, annotationIm[:, :, 1:3], layer_id, False, ["fat", "meat"]
    )
    hist_pairs.append(h)
    ax[layer_id].plot(h[0:80, :])
plt.show()


# %%
hist_pairs

# %% [markdown]
# #### Is it a reasonable assumption that each pixel is either meat or fat?
# I think mostly it is a reasonable assumption, though I am indeed seeing that sometimes, fat is '
    seen' through a thin layer of meat atop.#
#

# %% [markdown]
# ### Threshold value for a single spectral band
#
# 1. Calculate the threshold value t for all spectral bands for day 1.
# 2. Calculate the error rate for each spectral band.
# 3. Identify the spectral band, which has the best discriminative properties for meat and fat.
# 4. Classify the entire image of the salami for day 1, and visualise it.

# %% [markdown]
# #### Helper functions

# %%
rgbOut.shape

# %%
def LDA_discriminant(mu, var, prior):
    return lambda x: x * mu / var - mu**2 / (var * 2) + np.log(prior)


def LDA_classifier(mu1, var1, prior1, mu2, var2, prior2):
    disc1 = LDA_discriminant(mu1, var1, prior1)
    disc2 = LDA_discriminant(mu2, var2, prior2)
    return lambda x: 1 if disc1(x) > disc2(x) else 0


def class_hat_mean(x):
    return np.mean(x)


def pooled_hat_var(x, y):
    x_hat_var = np.var(x, axis=0)
    y_hat_var = np.var(y, axis=0)
    n_x = len(x)
    n_y = len(y)
```

```
    return ((n_x - 1) * x_hat_var + (n_y - 1) * y_hat_var) / (n_x + n_y - 2)


def prior(x):
    total = 514 * 514
    return x.shape[0] / total


### test for day one

# %%
multiIm1, annotationIm1 = hf.loadMulti(
    "multispectral_day01.mat", "annotation_day01.png", dirIn
)
fatPix1, fatR1, fatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 1])
meatPix1, meatR1, meatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 2])

plt.imshow(plt.imread("./data/color_day01.png"))
### for day one
fatPix1l1 = fatPix1[:, 0]
meatPix1l1 = meatPix1[:, 0]
layer_1 = multiIm1[:, :, 0]

fat_mean_1l1 = class_hat_mean(fatPix1l1)
meat_mean_1l1 = class_hat_mean(meatPix1l1)

fat_meat_pooled_var_1l1 = pooled_hat_var(meatPix1l1, fatPix1l1)

prior_fat_1l1 = prior(fatPix1l1)
prior_meat_1l1 = prior(meatPix1l1)

LDA_1l1 = LDA_classifier(
    fat_mean_1l1,
    fat_meat_pooled_var_1l1,
    prior_fat_1l1,
    meat_mean_1l1,
    fat_meat_pooled_var_1l1,
    prior_meat_1l1,
)

plt.figure()
plt.imshow(layer_1, cmap="gray")
plt.show()


# %%
labeled_layer_1 = np.zeros((multiIm1.shape[0], multiIm1.shape[1]))

for i in range(layer_1.shape[0]):
    for j in range(layer_1.shape[1]):
        labeled_layer_1[i, j] = LDA_1l1(layer_1[i, j])
plt.figure()
plt.imshow(labeled_layer_1, cmap="gray")
plt.show()

# %%
labeled_layer_1[*[2, 3]]

# %% [markdown]
```

```
# #### All steps in one go

# %%
### Calculate the accuracy of the classifier for layer 1 on the known pixels
fatPixId1 = np.stack((fatR1, fatC1), axis=1)
meatPixId1 = np.stack((meatR1, meatC1), axis=1)


def error_rate(labeled_image, fatPixId, meatPixId):

    correct = 0
    total = fatPixId.shape[0] + meatPixId.shape[0]
    for pix in fatPixId:
        pred_val = labeled_image[*pix]
        correct += pred_val == 1
    for pix in meatPixId:
        pred_val = labeled_image[*pix]
        correct += pred_val == 0
    # print("accuracy:", correct / total)
    # print("error rate:", 1 - correct / total)
    return 1 - correct / total


error_rate(labeled_layer_1, fatPixId1, meatPixId1)

# %%
error_rates = []
for layer_id in range(n_layers):
    ### for day one
    fatPix1l = fatPix1[:, layer_id]
    meatPix1l = meatPix1[:, layer_id]
    layer = multiIm1[:, :, layer_id]

    fat_mean_1l = class_hat_mean(fatPix1l)
    meat_mean_1l = class_hat_mean(meatPix1l)

    fat_var_1l = pooled_hat_var(fatPix1l, meatPix1l)
    meat_var_1l = pooled_hat_var(meatPix1l, fatPix1l)

    prior_fat_1l = prior(fatPix1l)
    prior_meat_1l = prior(meatPix1l)

    LDA_1l = LDA_classifier(
        fat_mean_1l,
        fat_var_1l,
        prior_fat_1l,
        meat_mean_1l,
        meat_var_1l,
        prior_meat_1l,
    )
    labeled_layer = np.zeros((multiIm1.shape[0], multiIm1.shape[1]))

    for i in range(layer.shape[0]):
        for j in range(layer.shape[1]):
            labeled_layer[i, j] = LDA_1l(layer[i, j])
    # plt.figure()
    # plt.imshow(labeled_layer, cmap="gray")
    # plt.show()

```

```
    error_rates.append(error_rate(labeled_layer, fatPixId1, meatPixId1))

# %% [markdown]
# #### Identify the spectral band, which has the best discriminative properties for meat and fat.

# %%
error_rates

# %%
# compute basic statistics for the error rates
mean_error_rate = np.mean(error_rates)
std_error_rate = np.std(error_rates)
print(f"Mean error rate: {mean_error_rate:.4f}")
print(f"Standard deviation of error rates: {std_error_rate:.4f}")

# %%
error_rates[13]

# %%
print(np.min(error_rates))
print(np.argmin(error_rates))
best_layer_id = np.argmin(error_rates)

# %% [markdown]
# #### Majority baseline

# %%
# Majority baseline for day one
# Predict the majority class (meat or fat) for all annotated pixels

# Count annotated pixels for each class
num_fat = fatPix1.shape[0]
num_meat = meatPix1.shape[0]

# Determine majority class
majority_class = 1 if num_fat > num_meat else 0 # 1: fat, 0: meat

# Create a labeled image with majority class for all salami pixels
majority_labeled_layer = np.zeros((multiIm1.shape[0], multiIm1.shape[1]))
salami_region = annotationIm1.max(axis=2).astype(bool)
majority_labeled_layer[salami_region] = majority_class

# Calculate error rate for majority baseline
fatPixId1 = np.stack((fatR1, fatC1), axis=1)
meatPixId1 = np.stack((meatR1, meatC1), axis=1)
majority_error = error_rate(majority_labeled_layer, fatPixId1, meatPixId1)

print(f"Majority class: {'fat' if majority_class == 1 else 'meat'}")
print(f"Majority baseline error rate: {majority_error}")

# Visualize majority baseline
plt.figure()
plt.imshow(majority_labeled_layer, cmap="gray")
plt.title("Majority Baseline Classification (Day 1)")
plt.show()

# %% [markdown]
# #### Classify the entire image of the salami for day 1, and visualise it.

```

```
# %%
### for day one
fatPix1l = fatPix1[:, best_layer_id]
meatPix1l = meatPix1[:, best_layer_id]
layer = multiIm1[:, :, best_layer_id]

fat_mean_1l = class_hat_mean(fatPix1l)
meat_mean_1l = class_hat_mean(meatPix1l)

fat_meat_pooled_var_1l = pooled_hat_var(fatPix1l, meatPix1l)

prior_fat_1l = prior(fatPix1l)
prior_meat_1l = prior(meatPix1l)

LDA_1l_best = LDA_classifier(
    fat_mean_1l,
    fat_meat_pooled_var_1l,
    prior_fat_1l,
    meat_mean_1l,
    fat_meat_pooled_var_1l,
    prior_meat_1l,
)
labeled_layer = np.zeros((multiIm1.shape[0], multiIm1.shape[1]))

for i in range(layer.shape[0]):
    for j in range(layer.shape[1]):
        labeled_layer[i, j] = LDA_1l_best(layer[i, j])
plt.figure()
plt.imshow(labeled_layer, cmap="gray")
plt.show()

error_rate(labeled_layer, fatPixId1, meatPixId1)


# %% [markdown]
# ### Threshold value for a single spectral band
# 1. Calculate the threshold value t for all spectral bands for day 1.
# 2. Calculate the error rate for each spectral band.
# 3. Identify the spectral band, which has the best discriminative properties for meat and fat.
# 4. Classify the entire image of the salami for day 1, and visualise it.

# %%
def class_md_hat_mean(X):
    """Assumes observations in X are in rows, and dimensions in columns"""
    return np.mean(X, axis=0)


def class_hat_covar(X, X_hat_mean=None):
    """Observations are in row. Computes covariance matrix"""
    if X_hat_mean is None:
        X_hat_mean = class_md_hat_mean(X)
    X_c = X - X_hat_mean
    n = X.shape[0]
    return 1 / (n - 1) * X_c.T @ X_c


def pool_hat_covar(X, Y):
    """Observations are in row. Computes pooled covariance matrix"""
    X_hat_covar = class_hat_covar(X)
```

```
    Y_hat_covar = class_hat_covar(Y)
    n_X = X.shape[0]
    n_Y = Y.shape[0]
    return ((n_X - 1) * X_hat_covar + (n_Y - 1) * Y_hat_covar) / (n_X + n_Y - 2)


def prior(x):
    total = 514 * 514
    return x.shape[0] / total


### test for day one


def md_LDA_discriminant(X_hat_mean, X_pool_covar, prior):
    """Multi dimensional, observation provided as a row"""
    inv_covar = np.linalg.inv(X_pool_covar)
    return lambda x: (
        x @ inv_covar @ X_hat_mean.T
        - 1 / 2 * X_hat_mean @ inv_covar @ X_hat_mean.T
        + np.log(prior)
    )


def md_LDA_classifier(X_hat_mean_1, X_hat_mean_2, prior_1, prior_2, X_pool_hat_covar):
    disc1 = md_LDA_discriminant(X_hat_mean_1, X_pool_hat_covar, prior_1)
    disc2 = md_LDA_discriminant(X_hat_mean_2, X_pool_hat_covar, prior_2)
    # print(disc1(x), disc2(x))
    return lambda x: 1 if disc1(x) > disc2(x) else 0


# %%
import helper_functions.helpFunctions as hf
import matplotlib.pyplot as plt
import numpy as np

dirIn = "./data/"


# %%
fatPix.shape

# %%
# multivariate case


multiIm1, annotationIm1 = hf.loadMulti(
    "multispectral_day01.mat", "annotation_day01.png", dirIn
)
fatPix1, fatR1, fatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 1])
meatPix1, meatR1, meatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 2])

plt.imshow(plt.imread("./data/color_day01.png"))
### for day one

fat_mean_1 = class_md_hat_mean(fatPix1)
meat_mean_1 = class_md_hat_mean(meatPix1)

fat_mean_pooled_covar = pool_hat_covar(fatPix1, meatPix1)
```

```
485
486
487 prior_fat_1 = prior(fatPix1)
488 prior_meat_1 = prior(meatPix1)
489
490 LDA_1 = md_LDA_classifier(
491     fat_mean_1,
492     meat_mean_1,
493     prior_fat_1,
494     prior_meat_1,
495     fat_mean_pooled_covar,
496 )
497
498 labeled_multiIm1 = np.zeros((multiIm1.shape[0], multiIm1.shape[1]))
499
500 for i in range(multiIm1.shape[0]):
501     for j in range(multiIm1.shape[1]):
502         labeled_multiIm1[i, j] = LDA_1(multiIm1[i, j])
503 plt.figure()
504 plt.imshow(labeled_multiIm1, cmap="gray")
505 plt.show()
506
507 error_rate(labeled_multiIm1, fatPixId1, meatPixId1)
508
509 # %%
510 error_rate(labeled_multiIm1, fatPixId1, meatPixId1)
511
512 # %%
513 fatR1
514
515 # %% [markdown]
516 # # Calculation For All Days
517 # 1. Classify fat and meat for the remaining days with the models trained on day 1.
518 # 2. Calculate the error rate for the annotated areas for the remaining days. How is the
      performance of
519 # the two models?
520 # 3. Classify again the entire images for the remaining days and, and visualise them. Judged on the
521 # visualisations, which model performs best?
522
523 # %%
524 ### Code Plan
525
526 ### Load data
527 from tornado.gen import multi
528
529 test_days = ["01", "06", "13", "20", "28"]
530 test_days_data = {}
531 for day in test_days:
532     multiIm, annotationIm = hf.loadMulti(
533         f"multispectral_day{day}.mat", f"annotation_day{day}.png", dirIn
534     )
535     fat_annots = hf.getPix(multiIm, annotationIm[:, :, 1])
536     meat_annots = hf.getPix(multiIm, annotationIm[:, :, 2])
537     test_days_data[day] = {
538         "multiIm": multiIm,
539         "annotationIm": annotationIm,
540         "fat_annots": fat_annots,
541         "meat_annots": meat_annots,
542     }
```

```
# fatPix1, fatR1, fatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 1])
# meatPix1, meatR1, meatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 2])


### Iterate over the data
for day, data in test_days_data.items():
    multiIm = data["multiIm"]
    fatPix, fatR, fatC = data["fat_annots"]
    meatPix, meatR, meatC = data["meat_annots"]
    #### Create empty layers
    labeled_layer_one_dim = np.zeros((multiIm.shape[0], multiIm.shape[1]))
    labeled_layer_multi_dim = labeled_layer_one_dim.copy()
    #### laber the layer using both classifiers (define above) - LDA_1 and LDA_1l
    best_layer = multiIm[:, :, best_layer_id]
    for i in range(best_layer.shape[0]):
        for j in range(best_layer.shape[1]):
            labeled_layer_one_dim[i, j] = LDA_1l_best(best_layer[i, j])

    for i in range(multiIm.shape[0]):
        for j in range(multiIm.shape[1]):
            labeled_layer_multi_dim[i, j] = LDA_1(multiIm[i, j])
    #### Compute the error rates for both
    fatPixId = np.stack((fatR, fatC), axis=1)
    meatPixId = np.stack((meatR, meatC), axis=1)
    er_om = error_rate(labeled_layer_one_dim, fatPixId, meatPixId)
    print(
        f"Day {day} - One Dimensional LDA Accuracy: {1 - er_om} - Error Rate: {er_om}"
    )
    er_md = error_rate(labeled_layer_multi_dim, fatPixId, meatPixId)
    print(
        f"Day {day} - Multi Dimensional LDA Accuracy: {1 - er_md} - Error Rate: {er_md}"
    )
    #### Display the annotation and the annotations from two classifiers, in one row - print days.
    color_im = plt.imread(f"./data/color_day{day}.png")
    fig, ax = plt.subplots(1, 3, figsize=(15, 5))
    ax[0].imshow(color_im)
    ax[0].set_title(f"Day {day} - Color Image")
    ax[1].imshow(labeled_layer_one_dim, cmap="gray")
    ax[1].set_title(f"Day {day} - One Dimensional LDA")
    ax[2].imshow(labeled_layer_multi_dim, cmap="gray")
    ax[2].set_title(f"Day {day} - Multi Dimensional LDA")
    plt.show()

# %%
### Multi looks better!

# %% [markdown]
# Training on each day and error calculated on all the other days:
# 1. Judging from the images, which day would you choose to train on and why??
# 2. Train the linear discriminant function on day 1, day 6 and so on, and for each day calculate
    the
# error rate on all the other days.
# 3. Show the error rate for all days (5 training days x 4 test days = 20 error rates) in an
    appropriate
# plot or table.
# 4. Why do we exclude the day we have trained on from the comparison?
# 5. Which day is the best to train the model on, and why?
# 6. What are the error sources in the model? Can we trust that the calculated fat and meat content
     in
```

```
# the salami is the same as the real content?

# %% [markdown]
# 1. I think the day 13. We want to have the image that is the most representative of them all. Day 13 if the first day that contains clearly seen shadow, while it is also not too strong. I think it might be a good trade-off thus.
# 4. It does not make a sense to do it at all, simply to test and train on main

# %%
### Load data for all days (again, yes)


days = ["01", "06", "13", "20", "28"]
days_data = {}
for day in days:
    multiIm, annotationIm = hf.loadMulti(
        f"multispectral_day{day}.mat", f"annotation_day{day}.png", dirIn
    )
    fat_annots = hf.getPix(multiIm, annotationIm[:, :, 1])
    meat_annots = hf.getPix(multiIm, annotationIm[:, :, 2])
    days_data[day] = {
        "multiIm": multiIm,
        "annotationIm": annotationIm,
        "fat_annots": fat_annots,
        "meat_annots": meat_annots,
    }
# fatPix1, fatR1, fatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 1])
# meatPix1, meatR1, meatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 2])

### Create results table
error_rates_table = np.zeros((len(days), len(days)))

### For each day,
#### train the multivariate LDA on it
for day, data in days_data.items():
    multiIm = data["multiIm"]
    fatPix, fatR, fatC = data["fat_annots"]
    meatPix, meatR, meatC = data["meat_annots"]

    fat_mean = class_md_hat_mean(fatPix)
    meat_mean = class_md_hat_mean(meatPix)

    fat_mean_pooled_covar = pool_hat_covar(fatPix, meatPix)

    prior_fat = prior(fatPix)
    prior_meat = prior(meatPix)

    LDA_classifier_day = md_LDA_classifier(
        fat_mean,
        meat_mean,
        prior_fat,
        prior_meat,
        fat_mean_pooled_covar,
    )
    #### obtain error rate having evaluated on the remaining days
    for eval_day, eval_data in days_data.items():
        if eval_day == day:
            continue
        eval_multiIm = eval_data["multiIm"]
```

```
        eval_fatPix, eval_fatR, eval_fatC = eval_data["fat_annots"]
        eval_meatPix, eval_meatR, eval_meatC = eval_data["meat_annots"]

        labeled_eval_multiIm = np.zeros((eval_multiIm.shape[0], eval_multiIm.shape[1]))

        for i in range(eval_multiIm.shape[0]):
            for j in range(eval_multiIm.shape[1]):
                labeled_eval_multiIm[i, j] = LDA_classifier_day(eval_multiIm[i, j])

        eval_fatPixId = np.stack((eval_fatR, eval_fatC), axis=1)
        eval_meatPixId = np.stack((eval_meatR, eval_meatC), axis=1)

        er_eval_md = error_rate(labeled_eval_multiIm, eval_fatPixId, eval_meatPixId)

        error_rates_table[days.index(day), days.index(eval_day)] = er_eval_md

        print(
            f"Trained on Day {day} - Evaluated on Day {eval_day} - Multi Dimensional LDA Accuracy:
                {1 - er_eval_md} - Error Rate: {er_eval_md}"
        )
#### Put that all into a table TxT

# %%
### Display the table as a heatmap, with days on x and y axis, and error rates in the cells
import seaborn as sns

# have first diagonal as NaN, since we don't evaluate on the same day
np.fill_diagonal(error_rates_table, np.nan)

plt.figure(figsize=(10, 8))
sns.heatmap(
    error_rates_table,
    xticklabels=days,
    yticklabels=days,
    annot=True,
    fmt=".2f",
    cmap="YlGnBu",
)
plt.title("Error Rates of Multi Dimensional LDA Classifier")
plt.xlabel("Evaluation Day")
plt.ylabel("Training Day")
plt.show()

# %% [markdown]
# 5. I think it is the dain 13. It achieves similar performance to the day 28. I would not pick the
      day 28, because it is the last day - it might be overfitted too much to older salamis and this
      is just a luck it was better than 13. (if we are doing this without a statistical test)

# %%
plt.figure()
plt.imshow(annotationIm.max(axis=2), cmap="gray")
plt.figure()

# %%
from helper_functions.helpFunctions import setImagePix
### Load data for all days (again, yes)

best_train_day = "28"

```

```
712
713 days = ["01", "06", "13", "20", "28"]
714 days_data = {}
715 for day in days:
716     multiIm, annotationIm = hf.loadMulti(
717         f"multispectral_day{day}.mat", f"annotation_day{day}.png", dirIn
718     )
719     fat_annots = hf.getPix(multiIm, annotationIm[:, :, 1])
720     meat_annots = hf.getPix(multiIm, annotationIm[:, :, 2])
721     days_data[day] = {
722         "multiIm": multiIm,
723         "annotationIm": annotationIm,
724         "fat_annots": fat_annots,
725         "meat_annots": meat_annots,
726         "color_im": plt.imread(f"./data/color_day{day}.png"),
727     }
728 # fatPix1, fatR1, fatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 1])
729 # meatPix1, meatR1, meatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 2])
730
731 ### Create results table
732 # error_rates_table = np.zeros((len(days), len(days)))
733
734 data = days_data[best_train_day]
735
736 multiIm = data["multiIm"]
737 fatPix, fatR, fatC = data["fat_annots"]
738 meatPix, meatR, meatC = data["meat_annots"]
739
740 fat_mean = class_md_hat_mean(fatPix)
741 meat_mean = class_md_hat_mean(meatPix)
742
743 fat_mean_pooled_covar = pool_hat_covar(fatPix, meatPix)
744
745 prior_fat = prior(fatPix)
746 prior_meat = prior(meatPix)
747
748 LDA_classifier_day = md_LDA_classifier(
749     fat_mean,
750     meat_mean,
751     prior_fat,
752     prior_meat,
753     fat_mean_pooled_covar,
754 )
755 #### obtain error rate having evaluated on the remaining days
756 for day, data in days_data.items():
757     if day == best_train_day:
758         continue
759
760     multiIm = data["multiIm"]
761     fatPix, fatR, fatC = data["fat_annots"]
762     meatPix, meatR, meatC = data["meat_annots"]
763     salami_region = data["annotationIm"].max(axis=2).astype(bool)
764
765     labeled_multiIm = np.zeros((multiIm.shape[0], multiIm.shape[1]))
766
767     labeled_x, labeled_y = np.where(salami_region)
768
769     for i, j in zip(labeled_x, labeled_y):
770         labeled_multiIm[i, j] = (
```

```
            LDA_classifier_day(multiIm[i, j]) + 1
        ) # add one to differentiate from background

    fatPixId = np.stack((fatR, fatC), axis=1)
    meatPixId = np.stack((meatR, meatC), axis=1)

    er_md = error_rate(labeled_multiIm, fatPixId, meatPixId)

    masked_colour_image = setImagePix(data["color_im"], salami_region)

    ### meat mask
    meatOut = hf.setImagePix(data["color_im"], labeled_multiIm == 1)

    fatOut = hf.setImagePix(data["color_im"], labeled_multiIm == 2)

    plt.figure(figsize=(15, 5))
    plt.subplot(1, 3, 1)
    plt.imshow(masked_colour_image)
    plt.title(f"Day {day} - Original Image")
    plt.subplot(1, 3, 2)
    plt.imshow(meatOut)
    plt.title(f"Day {day} - Meat Mask")
    plt.subplot(1, 3, 3)
    plt.imshow(fatOut)
    plt.title(f"Day {day} - Fat Mask")
    plt.show()

    # print fat percentage
    fat_percentage = np.sum(labeled_multiIm == 2) / np.sum(salami_region)
    print(f"Day {day} - Fat Percentage: {fat_percentage:.2%}")
    print(f"Day {day} - Acc: {1 - er_md} ER: {er_md}")

#### Put that all into a table TxT

# %%
from helper_functions.helpFunctions import setImagePix
### Load data for all days (again, yes)

best_train_day = "28"

tmp = None
days = ["01", "06", "13", "20", "28"]
days_data = {}
for day in days:
    multiIm, annotationIm = hf.loadMulti(
        f"multispectral_day{day}.mat", f"annotation_day{day}.png", dirIn
    )
    fat_annots = hf.getPix(multiIm, annotationIm[:, :, 1])
    meat_annots = hf.getPix(multiIm, annotationIm[:, :, 2])
    days_data[day] = {
        "multiIm": multiIm,
        "annotationIm": annotationIm,
        "fat_annots": fat_annots,
        "meat_annots": meat_annots,
        "color_im": plt.imread(f"./data/color_day{day}.png"),
    }
# fatPix1, fatR1, fatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 1])
# meatPix1, meatR1, meatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 2])

```

```
### Create results table
# error_rates_table = np.zeros((len(days), len(days)))

data = days_data[best_train_day]

multiIm = data["multiIm"]
fatPix, fatR, fatC = data["fat_annots"]
meatPix, meatR, meatC = data["meat_annots"]

fat_mean = class_md_hat_mean(fatPix)
meat_mean = class_md_hat_mean(meatPix)

fat_mean_pooled_covar = pool_hat_covar(fatPix, meatPix)

prior_fat = prior(fatPix)
prior_meat = prior(meatPix)

LDA_classifier_day = md_LDA_classifier(
    fat_mean,
    meat_mean,
    prior_fat,
    prior_meat,
    fat_mean_pooled_covar,
)
#### obtain error rate having evaluated on the remaining days
for day, data in days_data.items():
    if day == best_train_day:
        continue

    multiIm = data["multiIm"]
    fatPix, fatR, fatC = data["fat_annots"]
    meatPix, meatR, meatC = data["meat_annots"]
    salami_region = data["annotationIm"].max(axis=2).astype(bool)

    labeled_multiIm = np.zeros((multiIm.shape[0], multiIm.shape[1]))

    labeled_x, labeled_y = np.where(salami_region)

    for i, j in zip(labeled_x, labeled_y):
        labeled_multiIm[i, j] = (
            LDA_classifier_day(multiIm[i, j]) + 1
        ) # add one to differentiate from background

    fatPixId = np.stack((fatR, fatC), axis=1)
    meatPixId = np.stack((meatR, meatC), axis=1)

    er_md = error_rate(labeled_multiIm, fatPixId, meatPixId)

    masked_colour_image = setImagePix(data["color_im"], salami_region)

    ### meat mask
    meatOut = hf.setImagePix(data["color_im"], labeled_multiIm == 1)

    fatOut = hf.setImagePix(data["color_im"], labeled_multiIm == 2)

    plt.figure(figsize=(15, 5))
    plt.subplot(1, 3, 1)
    plt.imshow(masked_colour_image)
    plt.title(f"Day {day} - Original Image")
```

```
    plt.subplot(1, 3, 2)
    plt.imshow(meatOut)
    plt.title(f"Day {day} - Meat Mask")
    plt.subplot(1, 3, 3)
    plt.imshow(fatOut)
    plt.title(f"Day {day} - Fat Mask")
    plt.show()

    # print fat percentage
    fat_percentage = np.sum(labeled_multiIm == 2) / np.sum(salami_region)
    print(f"Day {day} - Fat Percentage: {fat_percentage:.2%}")
    print(f"Day {day} - Acc: {1 - er_md} ER: {er_md}")
    tmp = fatOut

#### Put that all into a table TxT

# %% [markdown]
# 6. Clearly, for the first 3 days it seems to be doing good, however, day 28 and day 20 seems not doing that great (especially fat seems not to be taken into accounnt appropriately)
# "Error sources" in the model:
# * I believe it is not about investigation "why" this happens, but which pixels couse an error. In this case
# * These are not DARK pixels (at the edge).
# * These are the pixels at the 'meaty' parts.
# * We can see from error rates the model is doing good on the labelled data.
# * From the visual inspections - it seems to be still doing pretty on the pixels that we would think should be classified as fat.

# %% [markdown]
# #### Prior knowledge:
# **30% fat in batch of salamis - OK.**
# 1. Incorporate this information into your linear discriminant model
# 2. Does it change your estimates?

# %%
from helper_functions.helpFunctions import setImagePix
### Load data for all days (again, yes)

best_train_day = "28"

tmp2 = None
mci = None
days = ["01", "06", "13", "20", "28"]
days_data = {}
for day in days:
    multiIm, annotationIm = hf.loadMulti(
        f"multispectral_day{day}.mat", f"annotation_day{day}.png", dirIn
    )
    fat_annots = hf.getPix(multiIm, annotationIm[:, :, 1])
    meat_annots = hf.getPix(multiIm, annotationIm[:, :, 2])
    days_data[day] = {
        "multiIm": multiIm,
        "annotationIm": annotationIm,
        "fat_annots": fat_annots,
        "meat_annots": meat_annots,
        "color_im": plt.imread(f"./data/color_day{day}.png"),
    }
# fatPix1, fatR1, fatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 1])
# meatPix1, meatR1, meatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 2])
```

```
945
946 ### Create results table
947 # error_rates_table = np.zeros((len(days), len(days)))
948
949 data = days_data[best_train_day]
950
951 multiIm = data["multiIm"]
952 fatPix, fatR, fatC = data["fat_annots"]
953 meatPix, meatR, meatC = data["meat_annots"]
954
955 fat_mean = class_md_hat_mean(fatPix)
956 meat_mean = class_md_hat_mean(meatPix)
957
958 fat_mean_pooled_covar = pool_hat_covar(fatPix, meatPix)
959
960 prior_fat = 0.3
961 prior_meat = 1 - prior_fat
962
963 LDA_classifier_day = md_LDA_classifier(
964     fat_mean,
965     meat_mean,
966     prior_fat,
967     prior_meat,
968     fat_mean_pooled_covar,
969 )
970 #### obtain error rate having evaluated on the remaining days
971 for day, data in days_data.items():
972     if day == best_train_day:
973         continue
974
975     multiIm = data["multiIm"]
976     fatPix, fatR, fatC = data["fat_annots"]
977     meatPix, meatR, meatC = data["meat_annots"]
978     salami_region = data["annotationIm"].max(axis=2).astype(bool)
979
980     labeled_multiIm = np.zeros((multiIm.shape[0], multiIm.shape[1]))
981
982     labeled_x, labeled_y = np.where(salami_region)
983
984     for i, j in zip(labeled_x, labeled_y):
985         labeled_multiIm[i, j] = (
986             LDA_classifier_day(multiIm[i, j]) + 1
987         ) # add one to differentiate from background
988
989     fatPixId = np.stack((fatR, fatC), axis=1)
990     meatPixId = np.stack((meatR, meatC), axis=1)
991
992     er_md = error_rate(labeled_multiIm, fatPixId, meatPixId)
993
994     masked_colour_image = setImagePix(data["color_im"], salami_region)
995
996     ### meat mask
997     meatOut = hf.setImagePix(data["color_im"], labeled_multiIm == 1)
998
999     fatOut = hf.setImagePix(data["color_im"], labeled_multiIm == 2)
1000
1001     plt.figure(figsize=(15, 5))
1002     plt.subplot(1, 3, 1)
1003     plt.imshow(masked_colour_image)
```

```
    plt.title(f"Day {day} - Original Image")
    plt.subplot(1, 3, 2)
    plt.imshow(meatOut)
    plt.title(f"Day {day} - Meat Mask")
    plt.subplot(1, 3, 3)
    plt.imshow(fatOut)
    plt.title(f"Day {day} - Fat Mask")
    plt.show()

    # print fat percentage
    fat_percentage = np.sum(labeled_multiIm == 2) / np.sum(salami_region)
    print(f"Day {day} - Fat Percentage: {fat_percentage:.2%}")
    print(
        f"Day {day} - Multi Dimensional LDA Accuracy: {1 - er_md} - Error Rate: {er_md}"
    )
    tmp2 = fatOut
    mci = masked_colour_image
#### Put that all into a table TxT

# %%
# NOT MUCH

# %%
# Confusion matrix for the chosen model (last trained in previous cell) on remaining days
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Use the best_train_day and LDA_classifier_day from previous cell
all_true = []
all_pred = []

for day, data in days_data.items():
    if day == best_train_day:
        continue
    multiIm = data["multiIm"]
    fatPix, fatR, fatC = data["fat_annots"]
    meatPix, meatR, meatC = data["meat_annots"]
    salami_region = data["annotationIm"].max(axis=2).astype(bool)

    # Fat pixels (label 1)
    for r, c in zip(fatR, fatC):
        all_true.append(1)
        all_pred.append(LDA_classifier_day(multiIm[r, c]))
    # Meat pixels (label 0)
    for r, c in zip(meatR, meatC):
        all_true.append(0)
        all_pred.append(LDA_classifier_day(multiIm[r, c]))

cm = confusion_matrix(all_true, all_pred, labels=[0, 1])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["meat", "fat"])
disp.plot(cmap="Blues")
plt.title(f"Confusion Matrix for Model Trained on Day {best_train_day}")
plt.show()

# %%
# plot a diff between diff between tmp and tmp2, that is plot part of tmp where tmp2 is not fat.

tmp_mask = np.where(tmp == 255, 0, 1).min(axis=2)
tmp2_mask = np.where(tmp2 == 255, 0, 1).min(axis=2)

```

```
# %%
diff = tmp_mask - tmp2_mask
plt.imshow(diff)

# %%
masked = mci * diff[:, :, None]
masked = np.where(masked == 0, 255, masked)
plt.imshow(masked)

# %% [markdown]
# # Limitations of the model

# %% [markdown]
# ## Covariance matrix diff. heatmap

# %%
from helper_functions.helpFunctions import setImagePix
### Load data for all days (again, yes)

best_train_day = "28"

days = [best_train_day]
days_data = {}
for day in days:
    multiIm, annotationIm = hf.loadMulti(
        f"multispectral_day{day}.mat", f"annotation_day{day}.png", dirIn
    )
    fat_annots = hf.getPix(multiIm, annotationIm[:, :, 1])
    meat_annots = hf.getPix(multiIm, annotationIm[:, :, 2])
    days_data[day] = {
        "multiIm": multiIm,
        "annotationIm": annotationIm,
        "fat_annots": fat_annots,
        "meat_annots": meat_annots,
        "color_im": plt.imread(f"./data/color_day{day}.png"),
    }
# fatPix1, fatR1, fatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 1])
# meatPix1, meatR1, meatC1 = hf.getPix(multiIm1, annotationIm1[:, :, 2])

### Create results table
# error_rates_table = np.zeros((len(days), len(days)))

data = days_data[best_train_day]

multiIm = data["multiIm"]
fatPix, fatR, fatC = data["fat_annots"]
meatPix, meatR, meatC = data["meat_annots"]

# fat_mean_pooled_covar = pool_hat_covar(fatPix, meatPix)
fat_covar = class_hat_covar(fatPix)
meat_covar = class_hat_covar(meatPix)

# Plot both covariance matrices in one figure
plt.figure(figsize=(16, 6))
plt.subplot(1, 2, 1)
sns.heatmap(fat_covar, fmt=".2f", cmap="YlGnBu")
plt.title("Covariance Matrix for Fat Class (Day 28)")
plt.xlabel("Spectral Band")
plt.ylabel("Spectral Band")
```

```
plt.subplot(1, 2, 2)
sns.heatmap(meat_covar, fmt=".2f", cmap="YlGnBu")
plt.title("Covariance Matrix for Meat Class (Day 28)")
plt.xlabel("Spectral Band")
plt.ylabel("Spectral Band")

plt.tight_layout()
plt.show()
```